# Story App (Storydico by Arafin) Dicoding 5-Star Plan

Comprehensive Step-by-Step Implementation Guide & Architectural Blueprint

> 📌 **Project Overview**
>
> This document provides the full step-by-step roadmap for developing the **Story App** Android application. The implementation strict satisfies all Dicoding Submission criteria while targeting the maximum **5-Star Rating** by incorporating all 8 recommended best-practice features.

## 1. System Architecture & Directory Structure

The app follows the **MVVM (Model-View-ViewModel)** architecture with clean separation of concerns, LiveData, ViewBinding, Kotlin Coroutines, and DataStore Preferences.

```
com.example.storydicobyarafin
├── data/
│   ├── api/
│   │   ├── ApiConfig.kt           // Retrofit instance + Auth Bearer token interceptor
│   │   ├── ApiService.kt          // Retrofit HTTP endpoints (Auth, Stories)
│   │   └── response/              // Data response objects (Login, Register, Story)
│   ├── pref/
│   │   ├── UserModel.kt           // User session data class
│   │   └── UserPreference.kt      // DataStore for session & token persistence
│   └── repository/
│       ├── UserRepository.kt      // Auth & Session management
│       └── StoryRepository.kt     // Story data fetching & uploading
├── di/
│   └── Injection.kt               // Service Locator for DI
├── ui/
│   ├── adapter/
│   │   └── StoryAdapter.kt        // RecyclerView Adapter for story list
│   ├── customview/
│   │   ├── EmailEditText.kt       // Custom view with real-time email validation
│   │   └── PasswordEditText.kt    // Custom view with real-time password length validation (< 8
chars)
│   ├── addstory/
│   │   ├── AddStoryActivity.kt    // Camera / Gallery upload screen
│   │   └── AddStoryViewModel.kt
│   ├── detail/
│   │   └── DetailStoryActivity.kt // Story detail view with Shared Element transition
│   ├── login/
│   │   ├── LoginActivity.kt       // Login screen with Property Animation
│   │   └── LoginViewModel.kt
│   ├── main/
│   │   ├── MainActivity.kt        // Story list screen + Logout
│   │   └── MainViewModel.kt
│   ├── register/
│   │   ├── RegisterActivity.kt    // Registration screen
│   │   └── RegisterViewModel.kt
│   └── ViewModelFactory.kt        // Factory providing repositories to ViewModels
└── utils/
    ├── Event.kt                   // Single LiveEvent wrapper
```

```
    ├── ResultState.kt            // Sealed class (Loading, Success, Error)
    └── Utils.kt                  // Image compression, Uri converter, Date formatter
```

## 2. Step-by-Step Implementation Guide

### Step 1: Build Setup & Dependencies

- Enable ViewBinding in `app/build.gradle.kts`: `buildFeatures { viewBinding = true }`
- Add required libraries:
  - **Networking**: Retrofit 2, Moshi / Gson Converter, OkHttp Logging Interceptor.
  - **Async & State**: Kotlin Coroutines, ViewModel KTX, LiveData KTX.
  - **Storage**: AndroidX DataStore Preferences.
  - **Image Handling**: Glide, CameraX (`camera-camera2`, `camera-lifecycle`, `camera-view`).
- Declare permissions in `AndroidManifest.xml`: `INTERNET`, `CAMERA`, and FileProvider for camera output.

### Step 2: Custom View EditTexts (Instant Validation)

- **`EmailEditText`**:
  - Extends `AppCompatEditText`.
  - Uses `addTextChangedListener` to monitor input instantly.
  - Validates email format with `Patterns.EMAIL_ADDRESS`. Shows error directly on field if invalid format.
- **`PasswordEditText`**:
  - Extends `AppCompatEditText`.
  - Monitors text length. If length < 8 characters, displays instant error (`R.string.error_password_min_8`) without needing button click.

### Step 3: Data Layer & Session Management

- **`UserModel`**: Holds user details (`name`, `userId`, `token`, `isLogin`).
- **`UserPreference`**:
  - Uses `DataStore<Preferences>` to asynchronously persist token and login session.
  - Provides `getSession()`, `saveSession()`, and `logout()` methods.
- **`ApiConfig`** & **`ApiService`**:
  - Configures Retrofit with OkHttp header interceptor adding `Authorization: Bearer <token>` dynamically.
  - Defines endpoints: `/register`, `/login`, `GET /stories`, `POST /stories` (Multipart photo + description).

### Step 4: Repository, Sealed Result State & ViewModels

- `ResultState<T>` : Sealed class handling `Loading` , `Success(data)` , and `Error(message)` .
- `UserRepository` & `StoryRepository` : Clean data layer encapsulating network requests and session state.
- `ViewModelFactory` : Service Locator pattern supplying single repositories to ViewModels.

### Step 5: Authentication Pages (Login & Register)

- `LoginActivity` (IDs: `ed_login_email` , `ed_login_password` ):
  - Checks session on launch. If already logged in, immediately navigates to `MainActivity` and finishes.
  - **Property Animation**: Smooth staggered entrance animation for logo, text fields, and login button using `ObjectAnimator` and `AnimatorSet` .
  - Observes login state LiveData. Navigates to main screen *only on successful API response*.
- `RegisterActivity` (IDs: `ed_register_name` , `ed_register_email` , `ed_register_password` ):
  - Registration form with custom input validation.
  - Navigates to login screen upon successful registration response.

### Step 6: Story List Screen & Detail Screen

- `MainActivity` :
  - Displays list of stories using `RecyclerView` and `StoryAdapter` .
  - Item views display User Name ( `R.id.tv_item_name` ) and Photo ( `R.id.iv_item_photo` ).
  - Includes top app bar menu item for **Logout** ( `R.id.action_logout` ). Clicking clears session token and redirects to `LoginActivity` with cleared backstack.
  - Handles Loading ProgressBar, Error Snackbar, and Empty list placeholder.
- `DetailStoryActivity` :
  - Displays detailed story info: Name ( `R.id.tv_detail_name` ), Photo ( `R.id.iv_detail_photo` ), and Description ( `R.id.tv_detail_description` ).
  - **Shared Element Transition**: Smooth animated transition between list item photo/name and detail photo/name using `ActivityOptionsCompat.makeSceneTransitionAnimation` .

### Step 7: Add Story Screen (Camera, Gallery & Upload)

- `AddStoryActivity` :
  - Accessible via FAB / Add action on main screen.
  - Provides options for picking photo from Gallery or taking a photo using Camera / CameraX.
  - Description input box ( `R.id.ed_add_description` ).
  - Upload button ( `R.id.button_add` ).
  - **Image Compression**: Reduces file size to ≤ 1MB before sending `MultipartBody.Part` to API.
  - On upload success, returns to `MainActivity` with new story shown at top.

### Step 8: Localization, UI Flow & Student Note

- **Localization**: Strings provided in both English ( `values/strings.xml` ) and Indonesian ( `values-id/strings.xml` ).
- **Navigation Stack**: Correct intent flags ( `FLAG_ACTIVITY_NEW_TASK | FLAG_ACTIVITY_CLEAR_TASK` ) to ensure pressing Back on Home exits the app instead of returning to Auth/Upload screens.
- **Student Note**: Included in source documentation detailing property animation and shared element transitions used.

## 3. Dicoding 5-Star Criteria Compliance Matrix

| # | Dicoding Rating Suggestion / Criteria | Implementation Detail | Status |
|---|---|---|---|
| 1 | Clean Code & Conventions | MVVM pattern, clean imports, proper indentation, zero dead comments | ✓ **Included** |
| 2 | Custom View Email Validation | `EmailEditText` with instant format check without needing button click | ✓ **Included** |
| 3 | Camera & Gallery Photo Options | CameraX / Camera Intent + Photo Gallery Picker in `AddStoryActivity` | ✓ **Included** |
| 4 | Response-based Screen Navigation | Screen transition strictly triggered on successful API response | ✓ **Included** |
| 5 | Proper App Back Stack Navigation | Pressing Back on Home exits the app after login, upload, or logout | ✓ **Included** |
| 6 | Localization (Multi-language) | Full support for English & Indonesian ( `values` & `values-id` ) | ✓ **Included** |
| 7 | API Feedback & UI Interaction States | ProgressBar loading indicator, error Toast/Snackbar, empty state view | ✓ **Included** |
| 8 | MVVM Architecture & LiveData | ViewModel, LiveData, and Repository applied across all screens | ✓ **Included** |